

Attaque d'une base de données MySQL

Objectifs

Au cours de ces travaux pratiques, vous aurez accès à un fichier PCAP provenant d'une précédente attaque contre une base de données SQL.

Partie 1: Ouvrir Wireshark et charger le fichier PCAP.

Partie 2: Examiner l'attaque par Injection SQL.

Partie 3: L'attaque par injection SQL continue...

Partie 4: L'attaque par injection SQL fournit des informations système.

Partie 5: L'attaque par injection SQL et les informations des tables.

Partie 6: L'attaque par injection SQL se termine.

Contexte/scénario

Les attaques par injection SQL permettent aux hackers de taper des instructions SQL dans un site web et de recevoir une réponse de la base de données. Ils peuvent ainsi altérer les données se trouvant dans la base de données, usurper des identités et commettre divers méfaits.

Un fichier PPCE a été créé pour que vous puissiez voir une précédente attaque contre une base de données SQL. Vous allez également voir des attaques de base de données SQL et répondre aux questions.

Ressources requises

= Machine virtuelle Security Workstation

Instructions

Utilisez Wireshark, un analyseur de paquets réseau courant, pour analyser le trafic réseau. Après avoir lancé Wireshark, ouvrez une capture réseau précédemment enregistrée pour étudier les différentes étapes d'une attaque par injection SQL contre une base de données SQL.

Partie 1: Ouvrir Wireshark et charger le fichier PCAP.

Vous pouvez ouvrir l'application Wireshark de différentes manières sur un poste de travail Linux.

- Démarrez la VM Security Workstation.
- Cliquez sur **Applications > CyberOPS > Wireshark** sur le bureau et accédez à l'application Wireshark.
- Dans l'application Wireshark, cliquez sur **Ouvrir** au milieu de l'application sous Fichiers.
- Parcourez le répertoire **/home/analyst/** et recherchez **lab.support.files**. Dans le répertoire **lab.support.files** et ouvrez le fichier **SQL_Lab.pcap**.

- e. Le fichier PCAP s'ouvre dans Wireshark et affiche le trafic réseau capturé. Ce fichier de capture couvre une période de 8 minutes (441 secondes), soit la durée de cette attaque par injection SQL.

Quelles sont les deux adresses IP impliquées dans cette attaque par injection SQL selon les informations affichées?

Les deux adresses IP impliquées sont:

10.0.2.4 (L'attaquant/Source)

10.0.2.15 (La cible/Destination)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.2.4	10.0.2.15	TCP	74	35614 → 80 [SYN]
2	0.000315	10.0.2.15	10.0.2.4	TCP	74	80 → 35614 [SYN, ...]
3	0.000349	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK]
4	0.000681	10.0.2.4	10.0.2.15	HTTP	654	POST /dvwa/login.php
5	0.002149	10.0.2.15	10.0.2.4	TCP	66	80 → 35614 [ACK]
6	0.005700	10.0.2.15	10.0.2.4	HTTP	430	HTTP/1.1 302 Found
7	0.005700	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK]
8	0.014383	10.0.2.4	10.0.2.15	HTTP	496	GET /dvwa/index.php
9	0.015485	10.0.2.15	10.0.2.4	HTTP	3107	HTTP/1.1 200 OK
10	0.015485	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK]
11	0.068625	10.0.2.4	10.0.2.15	HTTP	429	GET /dvwa/dvwa/csrf
12	0.070400	10.0.2.15	10.0.2.4	HTTP	1511	HTTP/1.1 200 OK
13	174.254430	10.0.2.4	10.0.2.15	HTTP	536	GET /dvwa/vulnerability
14	174.254581	10.0.2.15	10.0.2.4	TCP	66	80 → 35638 [ACK]
15	174.257989	10.0.2.15	10.0.2.4	HTTP	1861	HTTP/1.1 200 OK
16	220.490531	10.0.2.4	10.0.2.15	HTTP	577	GET /dvwa/vulnerability
17	220.490637	10.0.2.15	10.0.2.4	TCP	66	80 → 35640 [ACK]
18	220.493085	10.0.2.15	10.0.2.4	HTTP	1918	HTTP/1.1 200 OK
19	277.727722	10.0.2.4	10.0.2.15	HTTP	630	GET /dvwa/vulnerability
20	277.727871	10.0.2.15	10.0.2.4	TCP	66	80 → 35642 [ACK]
21	277.732200	10.0.2.15	10.0.2.4	HTTP	1955	HTTP/1.1 200 OK

▶ Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
 ▶ Ethernet II, Src: PcsCompu ca:e1:24 (08:00:27:ca:e1:24), Dst: PcsCompu_9f:48:a0 (08:00:27:9f:48:a0)
 ▶ Internet Protocol Version 4, Src: 10.0.2.4, Dst: 10.0.2.15
 ▶ Transmission Control Protocol, Src Port: 35614, Dst Port: 80, Seq: 0, Len: 0

Partie 2: Examiner l'attaque par Injection SQL.

Au cours de cette étape, vous allez visualiser le début d'une attaque.

- a. Dans la capture Wireshark, cliquez avec le bouton droit de la souris sur la ligne 13 et sélectionnez **Follow > HTTP Stream**. La ligne 13 a été choisie, car il s'agit d'une requête GET HTTP. Cette fonction permet de suivre le flux de données tel que la couche application le voit et mène au test des requêtes à la recherche d'injection de code SQL.

Le trafic de la source s'affiche en rouge. La source a envoyé une requête GET à l'hôte 10.0.2.15. En bleu, l'appareil de destination répond à la source.

- b. Dans le champ **Find** Rechercher , saisissez **1=1**. Cliquez sur **Find Next** (Trouver le suivant)..
- c. L'attaquant a entré une requête (1=1) dans une boîte de recherche UserID sur la cible 10.0.2.15 pour voir si l'application est vulnérable à l'injection SQL. Au lieu de répondre avec un message d'échec de connexion, l'application répond avec un enregistrement d'une base de données. Le hacker a donc vérifié que la base de données répond lorsqu'il tape une commande SQL. La chaîne de recherche 1=1 crée une instruction SQL qui sera toujours vraie. Dans l'exemple, quelle que soit la requête saisie dans le champ, elle sera toujours vraie.
- d. Fermez la fenêtre Follow HTTP Stream (Suivre le flux HTTP).
- e. Cliquez sur **Clear display filter** (Effacer le filtre) d'affichage pour afficher la totalité de la conversation Wireshark.

Partie 3: L'attaque par injection SQL continue...

Au cours de cette étape, vous allez voir la suite d'une attaque.

- a. Dans la capture Wireshark, cliquez avec le bouton droit de la souris sur la ligne 19, puis cliquez sur **Follow** (Suivre) > **HTTP Stream** (Flux http).
- b. Dans le champ **Find** (Rechercher), entrez **1=1**. Cliquez sur **Find Next** (Trouver le suivant).
- c. L'attaquant a saisi une requête (1' ou 1=1 union select database(), user()#) dans un champ de recherche UserID sur la cible 10.0.2.15. Au lieu de répondre avec un message d'échec de connexion, l'application répond avec les informations suivantes:

Le nom de la base de données est **dvwa** et l'utilisateur de la base de données est **root@localhost**. Plusieurs comptes d'utilisateurs sont également affichés.

- d. Fermez la fenêtre Follow HTTP Stream (Suivre le flux HTTP).
- e. Cliquez sur **Clear display filter** (Effacer le filtre) d'affichage pour afficher la totalité de la conversation Wireshark.

Partie 4: L'attaque par injection SQL fournit des informations système.

Le hacker continue et commence à cibler des informations plus précises.

- Dans la capture Wireshark, cliquez avec le bouton droit de la souris sur la ligne 22 et sélectionnez **Follow > HTTP Stream**. Le trafic source indiqué en rouge envoie la requête GET à l'hôte 10.0.2.15. En bleu, l'appareil de destination répond à la source.
- Dans le champ **Find** (Rechercher), entrez **1=1**. Cliquez sur **Find Next** (Trouver les suivants).
- L'attaquant a entré une requête (1' or 1=1 union select null, version ()) dans une boîte de recherche UserID (d'ID d'utilisateur) sur la cible 10.0.2.15 pour trouver l'identifiant de version. Remarquez comment l'identifiant de version à la fin de la sortie, juste avant le </pre> code HTML de fermeture.

Quelle est la version?

La version est: 5.7.12-0ubuntu1.1

```
<pre>ID: 1' or 1=1 union select null, version
()#<br />First name: admin<br />Surname: admin</pre><pre>ID: 1' or
1=1 union select null, version ()#<br />First name: Gordon<br /
>Surname: Brown</pre><pre>ID: 1' or 1=1 union select null, version
()#<br />First name: Hack<br />Surname: Me</pre><pre>ID: 1' or 1=1
union select null, version ()#<br />First name: Pablo<br />Surname:
Picasso</pre><pre>ID: 1' or 1=1 union select null, version ()#<br /
>First name: Bob<br />Surname: Smith</pre><pre>ID: 1' or 1=1 union
select null, version ()#<br />First name: <br />Surname:
5.7.12-0ubuntu1.1</pre>
```

- Fermez la fenêtre Follow HTTP Stream (Suivre le flux HTTP).
- Cliquez sur **Clear display filter** (Effacer le filtre) d'affichage pour afficher la totalité de la conversation Wireshark.

Partie 5: L'attaque par injection SQL et les informations sur les tables.

Le hacker sait qu'il y a de nombreuses tables SQL contenant beaucoup d'informations. Le hacker essaie de les trouver.

- Dans la capture Wireshark, cliquez avec le bouton droit sur la ligne 25 et sélectionnez **Follow > HTTP Stream**. La source est indiquée en rouge. L'appareil source a envoyé une requête GET à l'hôte 10.0.2.15. En bleu, l'appareil de destination répond à la source.
- Dans le champ **Find** (Rechercher), saisissez **Users** (les utilisateurs). Cliquez sur **Find Next** (Trouver le suivant).
- L'attaquant a saisi une requête (1'or 1=1 union select null, table_name from information_schema.tables#) dans un champ de recherche UserID (d'ID d'utilisateur) sur la cible 10.0.2.15 pour afficher toutes les tables de la base de données. Une sortie indiquant de nombreuses tables est générée, car le hacker a spécifié « null » sans plus de précision.

Quelle serait la commande modifiée de (1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users') ?

```
1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE
table_name='users'#
```

```
table name from information schema.tables#<br />First name: <br />
Surname: help topic</pre><pre>ID: 1' or 1=1 union select null,
table name from information schema.tables#<br />First name: <br />
Surname: innodb index stats</pre><pre>ID: 1' or 1=1 union select
```

- Fermez la fenêtre Follow HTTP Stream (fenêtre Suivre le flux HTTP).
- Cliquez sur **Clear display filter** (Effacer le filtre) d'affichage pour afficher la totalité de la conversation Wireshark.

Partie 6: L'attaque par injection SQL se termine.

L'attaque se termine ayant permis au hacker de récupérer le meilleur butin possible: les valeurs de hash des mots de passe.

- Dans la capture Wireshark, cliquez avec le bouton droit de la souris sur la ligne 28 et sélectionnez **Follow > HTTP Stream**. La source est indiquée en rouge. L'appareil source a envoyé une requête GET à l'hôte 10.0.2.15. En bleu, l'appareil de destination répond à la source.
- Cliquez sur **Find** (Rechercher) et tapez **1=1**. Recherchez cette entrée. Lorsque le texte est localisé, cliquez sur **Cancel** (Annuler) dans la zone de recherche du texte à trouver.

Le hacker a saisi une requête (1'or 1=1 union select user, password from users#) dans une boîte de recherche d'ID d'utilisateur sur la cible 10.0.2.15 pour extraire les noms d'utilisateurs et les valeurs de hash des mots de passe !

À quel utilisateur correspond la valeur de hash de mot de passe 8d3533d75ae2c3966d7e0d4fcc69216b?

La valeur de hachage 8d3533d75ae2c3966d7e0d4fcc69216b correspond à l'utilisateur: 1337.

```
union select user, password from users#<br />First name: 1337<br />>Surname: 8d3533d75ae2c3966d7e0d4fcc69216b</pre><pre>ID: 1' or 1=1
```

- En utilisant un site Web tel que <https://crackstation.net/>, copiez le hachage du mot de passe dans le craqueur de hachage de mot de passe et commencez à craquer.

Quel est le mot de passe en texte clair?

Le mot de passe en texte clair est : charley

Hash	Type	Result
8d3533d75ae2c3966d7e0d4fcc69216b	md5	charley

- Fermez la fenêtre Follow HTTP Stream (Suivre le flux HTTP). Fermez les fenêtres ouvertes.

Questions de réflexion

1. Quel est le risque de voir les plateformes utiliser le langage SQL ?

L'injection SQL représente le principal risque. Lorsqu'une application n'est pas protégée, un pirate peut introduire des instructions SQL dans un champ de texte afin de réaliser deux actions :

Dérobage de données : Obtenir des renseignements sensibles tels que les identifiants d'utilisateur et les hashages de mots de passe (comme nous l'avons fait dans le laboratoire !).

Effacement/Modification de données : Si l'intrus dispose de privilèges suffisants, il peut supprimer des tables ou altérer le contenu de la base de données.

Ce n'est pas SQL qui pose problème, c'est la manière dont l'application traite les entrées.

2. Naviguez sur Internet et effectuez une recherche sur "prévenir les attaques par injection SQL". Quelles sont les 2 méthodes ou mesures à appliquer pour empêcher les attaques par injection SQL ?

On dispose de deux techniques très performantes pour se protéger :

L'usage de Requêtes Paramétrées (ou Préparées) : c'est la solution la plus efficace. On distingue la requête SQL des informations de l'utilisateur. La base de données ne perçoit jamais l'entrée de l'utilisateur comme une commande, mais simplement comme du texte à rechercher ou à stocker.

Validation et Filtrage des Entrées : On doit constamment s'assurer que l'utilisateur fournit le bon format de données (par exemple, refuser les lettres si on attend un numéro). Si l'entrée inclut des caractères spéciaux (tels que '), il est nécessaire de les « échapper » pour éviter toute rupture de la requête.